

Universidad Tecnológica de Honduras

ASIGNATURA:
PROGRAMACION MOVIL I

TRABAJO:
Tarea 2.5

CATEDRÁTICO:
Ricardo Lagos

REALIZADO POR:

Carlos Roberto Cardenas Martínez (202430010599)

REALIZADO POR:
Repositorio: github.com/Kirklen0910/Exa1Pa1199

FECHA:
07 noviembre de 2026

CONTENIDO

1. Introducción
2. Rama de trabajo y solicitud de integración
3. Pantalla de laboratorio
4. Implementación con Volley
5. Implementación con OkHttp
6. Implementación con Retrofit
7. Endpoints de prueba en la API
8. Tiempos de respuesta
9. Escenarios de falla
10. Evidencias de los escenarios
11. Cuadro comparativo
12. Recomendación
13. Conclusiones

1. INTRODUCCIÓN

En la tarea 2.3 la aplicación de la Bitácora de campo consumió la API con Retrofit y en la tarea 2.4 se protegió con tokens JWT. En esta tarea individual se implementó la misma consulta, GET /api/registros, con otras dos librerías: Volley y OkHttp usado directamente. Las tres versiones se compararon con criterios medibles y se probaron en los escenarios de falla más comunes: API detenida, dispositivo en modo avión, tiempo de espera agotado, token inválido y error interno del servidor.

En todos los escenarios la aplicación mostró un mensaje claro al usuario y en ninguno se cerró inesperadamente.

2. RAMA DE TRABAJO Y SOLICITUD DE INTEGRACIÓN

El trabajo se realizó en la rama personal feature/comparativa-carlos-cardenas del repositorio del equipo, con commits separados por cada parte del trabajo, y se integró a la rama principal mediante un pull request revisado por otro integrante.

Tabla 1. Commits de la rama

N.º	Commit
1	Dependencia de Volley y endpoints de prueba en la API
2	Consulta de registros con Volley
3	Consulta de registros con OkHttp asíncrono
4	Consulta con Retrofit reutilizando el repositorio
5	Pantalla comparativa y acceso desde el menú

Pull request: _____

3. PANTALLA DE LABORATORIO

Se agregó ComparativaActivity, accesible desde el menú de la lista de registros. Permite elegir el endpoint, repetir la consulta cinco veces para obtener el promedio, ejecutar cada librería con su propio botón y alterar la firma del token para provocar un 401. Cada tarjeta muestra la cantidad de registros recibidos y el tiempo en milisegundos, o el error técnico y el mensaje que ve el usuario. Todas las ejecuciones se registran en Logcat con la etiqueta Comparativa.

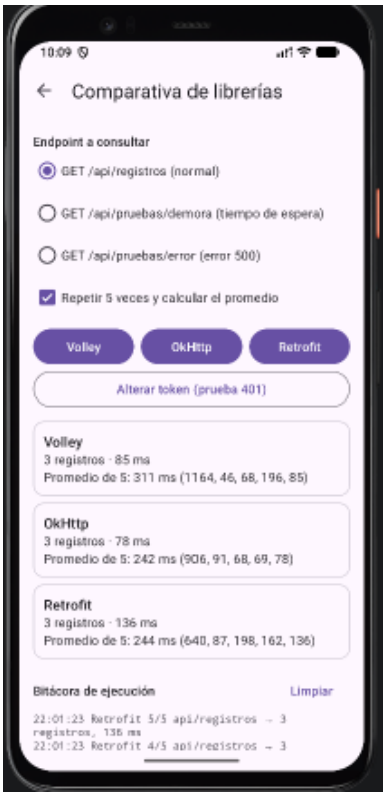


Figura 1. Pantalla de laboratorio con el resultado de las tres librerías

Tabla 2. Clases de la comparativa

Clase	Responsabilidad
ComparativaActivity	Pantalla, mide los tiempos, calcula el promedio y actualiza la interfaz.
ClienteVolley	Consulta con JsonArrayRequest, encabezados en getHeaders() y DefaultRetryPolicy.
ClienteOkHttp	Request construido manualmente, enqueue() y conversión con Gson.
ClienteRetrofit	Reutiliza RegistroRepository de la tarea 2.3.
ResultadoConsulta	Resultado común: cantidad, milisegundos, código HTTP, error técnico y mensaje.
RutaPrueba	Endpoint consultado: normal, demora o error 500.

4. IMPLEMENTACIÓN CON VOLLEY

ClienteVolley utiliza JsonArrayRequest. El token se envía sobrescribiendo getHeaders(), y DefaultRetryPolicy define un tiempo de espera de 10 segundos con un reintento. La cola de peticiones se crea una sola vez con el contexto de la aplicación.

```
JsonArrayRequest request = new JsonArrayRequest(Request.Method.GET, url, null,
        respuesta -> { ... }, error -> { ... }) {
    @Override
    public Map<String, String> getHeaders() {
        Map<String, String> encabezados = new HashMap<>();
        encabezados.put("Accept", "application/json");
        String token = sesion.getToken();
        if (token != null) encabezados.put("Authorization", "Bearer " + token);
        return encabezados;
    }
};
request.setRetryPolicy(new DefaultRetryPolicy(
        TIEMPO_ESPERA_MS, REINTENTOS, DefaultRetryPolicy.DEFAULT_BACKOFF_MULT));
cola.add(request);
```

La respuesta llega como JSONArray de org.json; se convierte a la lista de RegistroResumenDto con Gson. Los errores llegan como subclases de VolleyError: TimeoutError, NoConnectionError, AuthFailureError y ServerError, y el código HTTP se obtiene de networkResponse.statusCode. Las respuestas de Volley llegan en el hilo principal, por lo que la interfaz se actualiza directamente.

5. IMPLEMENTACIÓN CON OKHTTP

ClienteOkHttp construye el Request manualmente, agrega los encabezados y lo ejecuta de forma asíncrona con enqueue(). El cliente tiene tiempos de espera de 10 segundos para conexión, lectura y escritura.

```
Request.Builder builder = new Request.Builder()
        .url(BuildConfig.API_BASE_URL + ruta.getRuta())
        .header("Accept", "application/json")
        .get();
String token = sesion.getToken();
if (token != null) builder.header("Authorization", "Bearer " + token);

Call call = CLIENTE.newCall(builder.build());
call.enqueue(new Callback() {
    public void onFailure(Call c, IOException e) { ... }
    public void onResponse(Call c, Response response) {
        String json = response.body().string();
        List<RegistroResumenDto> registros = gson.fromJson(json, TIPO_LISTA);
    }
});
```

A diferencia de Volley, los métodos onResponse y onFailure se ejecutan en un hilo de OkHttp. Si se modificara la interfaz desde ahí, Android lanzaría CalledFromWrongThreadException. Por eso ComparativaActivity entrega el resultado al hilo principal con runOnUiThread:

```
llamadaOkHttp = clienteOkHttp.consultar(ruta,
        resultado -> runOnUiThread(() -> alTerminar.onResultado(resultado)));
```

OkHttp no considera error un 401 o un 500: llama a onResponse y el código se revisa con response.isSuccessful(). Solo los problemas de red llegan a onFailure, por ejemplo ConnectException o SocketTimeoutException.

6. IMPLEMENTACIÓN CON RETROFIT

ClienteRetrofit reutiliza RegistroRepository de la tarea 2.3, que llama a RegistroApi.listar(). El token lo agrega AuthInterceptor y ApiErrorMapper traduce las respuestas y excepciones a ApiError. Para los endpoints de prueba se agregó PruebasApi con una ruta variable (@Url) y el método obtenerRegistrosDesde() en el repositorio. Retrofit entrega la respuesta en el hilo principal.

```
return ruta == RutaPrueba.REGISTROS
        ? repositorio.obtenerRegistros(respuesta)
        : repositorio.obtenerRegistrosDesde(ruta.getRuta(), respuesta);
```

7. ENDPOINTS DE PRUEBA EN LA API

Para provocar el tiempo de espera y el error 500 se agregó temporalmente PruebasController. Ambos endpoints exigen token y solo responden en el entorno de desarrollo; en producción devuelven 404.

Tabla 3. Endpoints de prueba

Endpoint	Comportamiento
GET /api/pruebas/demora	Espera 30 segundos con await Task.Delay, más que el tiempo de espera de cualquier cliente.
GET /api/pruebas/error	Lanza InvalidOperationException; el middleware responde 500 en formato ProblemDetails.

```
[HttpGet("demora")]
public async Task<IActionResult> Demorar(CancellationToken ct)
{
    if (!_entorno.IsDevelopment()) return NotFound();
    await Task.Delay(Demora, ct);
    return Ok(Array.Empty<object>());
}
```

8. TIEMPOS DE RESPUESTA

Cada librería consultó GET /api/registros cinco veces seguidas en el emulador Pixel 4 con Android 17, con la API en el mismo equipo y tres registros en la base de datos.

Tabla 4. Tiempos de cinco ejecuciones (ms)

Librería	1	2	3	4	5	Promedio	Sin la 1.ª
Volley	1164	46	68	196	85	311.8	98.8
OkHttp	906	91	68	69	78	242.4	76.5
Retrofit	640	87	198	162	136	244.6	145.8

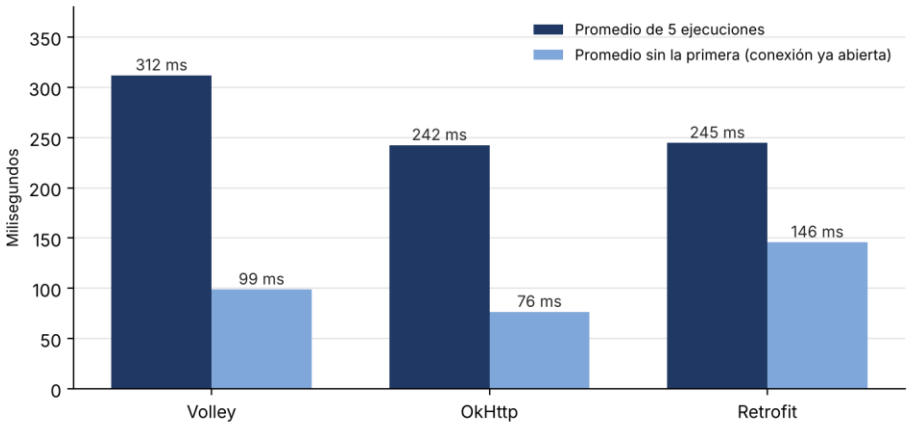


Figura 2. Promedio de tiempos de respuesta por librería

La primera ejecución de cada librería es la más lenta porque incluye abrir la conexión TCP y, en Volley, crear la cola de peticiones. Sin la primera, OkHttp fue la más rápida, seguida de Volley. Retrofit quedó algo más lento porque, en modo de depuración, HttpLoggingInterceptor escribe en Logcat el cuerpo completo de cada respuesta. Las diferencias son de pocas decenas de milisegundos, imperceptibles para el usuario.

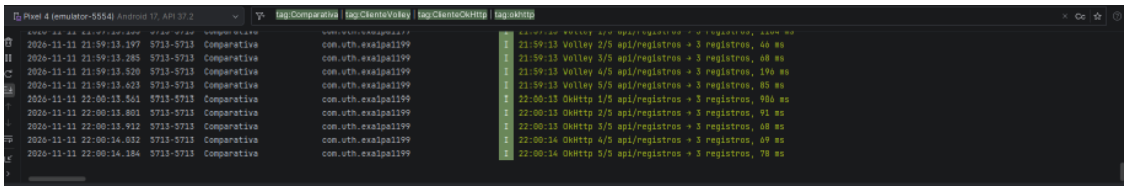


Figura 3. Logcat: cinco ejecuciones de Volley y de OkHttp

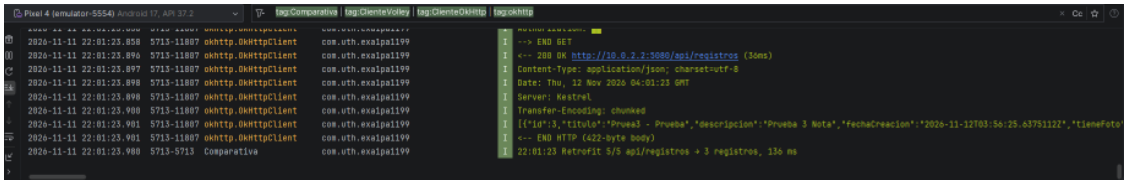


Figura 4. Logcat: última ejecución de Retrofit con el registro de OkHttp

9. ESCENARIOS DE FALLA

Se probaron las tres librerías en cada escenario. La Tabla 5 resume el error que recibe el código y la Tabla 6 el mensaje que ve el usuario. En ningún caso la aplicación se cerró.

Tabla 5. Error recibido por el código

Escenario	Volley	OkHttp	Retrofit
API detenida	NoConnectionError (2069 ms)	ConnectException, ECONNREFUSED (2062 ms)	ApiError.SIN_CONEXION, ConnectException (2143 ms)
Modo avión	NoConnectionError (73 ms)	ConnectException (60 ms)	ApiError.SIN_CONEXION

Escenario	Volley	OkHttp	Retrofit
Tiempo de espera	TimeoutError (30 553 ms, 1 reintento)	SocketTimeoutException (10 262 ms)	ApiError.TIEMPO_AGOTADO, SocketTimeoutException (20 535 ms)
Token alterado	AuthFailureError, HTTP 401 (44 ms)	onResponse con HTTP 401 Unauthorized (83 ms)	ApiError.NO_AUTENTICADO, HTTP 401
Error del servidor	ServerError, HTTP 500 (268 ms)	onResponse con HTTP 500 (1161 ms)	ApiError.SERVIDOR, HTTP 500 (132 ms)

Tabla 6. Mensaje que ve el usuario

Escenario	Mensaje
API detenida y modo avión	No se pudo conectar con el servidor. Revisa tu conexión o que la API esté encendida.
Tiempo de espera	El servidor tardó demasiado en responder. Inténtalo de nuevo.
Token alterado	Tu sesión expiró o no es válida. Inicia sesión de nuevo. Retrofit, además, borra la sesión mediante AuthInterceptor.
Error del servidor	El servidor tuvo un problema (código 500). Inténtalo más tarde.

El tiempo de espera muestra la diferencia de configuración: OkHttp abandonó a los 10 s; Retrofit, que usa el cliente de la tarea 2.3, a los 20 s; y Volley a los 30 s, porque DefaultRetryPolicy hizo un reintento y duplicó el tiempo de espera del segundo intento (10 s + 20 s). En la prueba 401 la firma del token se alteró desde la aplicación; la API la rechazó en las tres librerías. Como Retrofit cierra la sesión al recibir 401, se ejecutó al final.

10. EVIDENCIAS DE LOS ESCENARIOS

10.1 API detenida

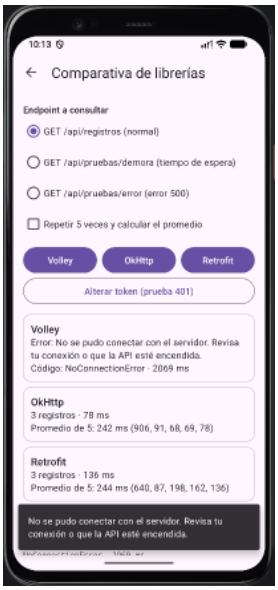


Figura 5. Volley: NoConnectionError



Figura 6. OkHttp: ConnectException

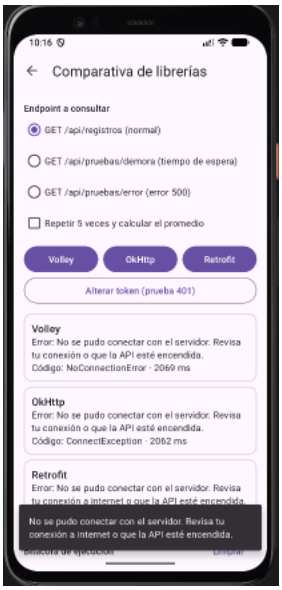


Figura 7. Retrofit: sin conexión

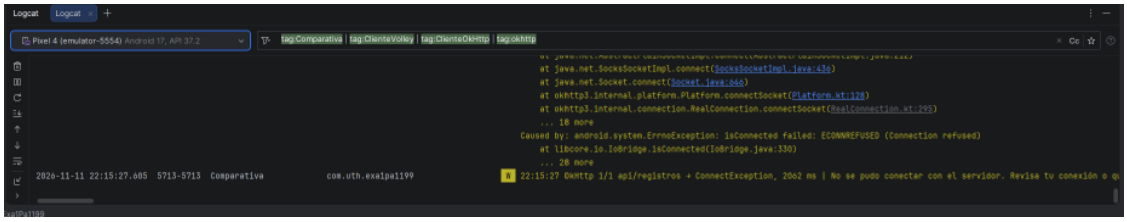


Figura 8. Logcat: OkHttp con ConnectException (Connection refused)

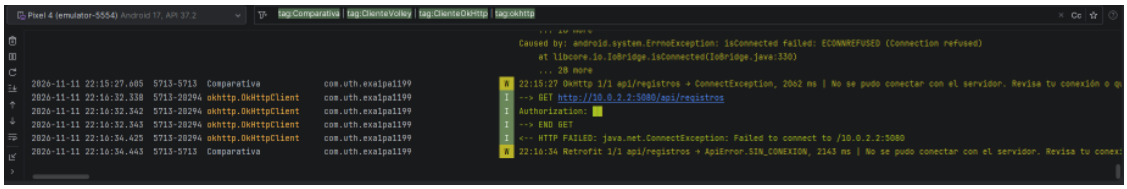


Figura 9. Logcat: Retrofit con ApiError.SIN_CONEXION

10.2 Dispositivo en modo avión

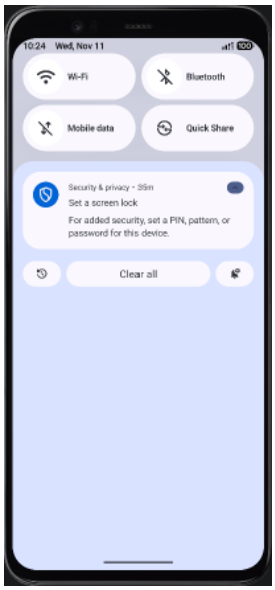


Figura 10. Modo avión activado

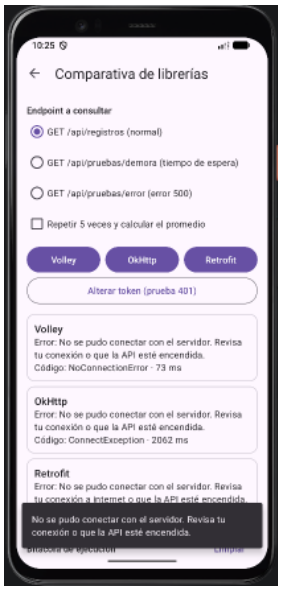


Figura 11. Volley: NoConnectionError

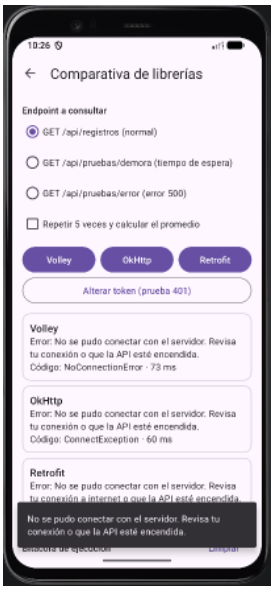


Figura 12. OkHttp: ConnectException

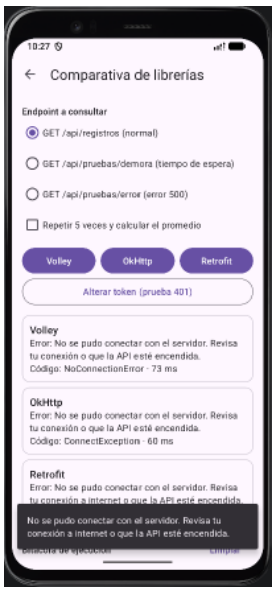


Figura 13. Retrofit: sin conexión

10.3 Tiempo de espera agotado

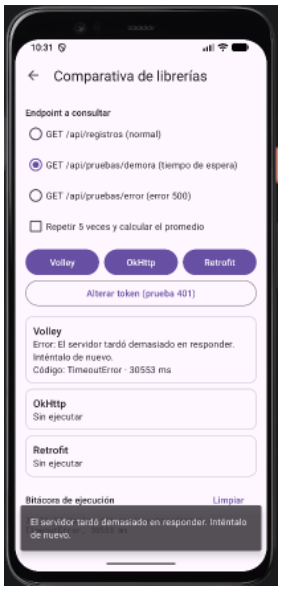


Figura 14. Volley: TimeoutError, 30 553 ms

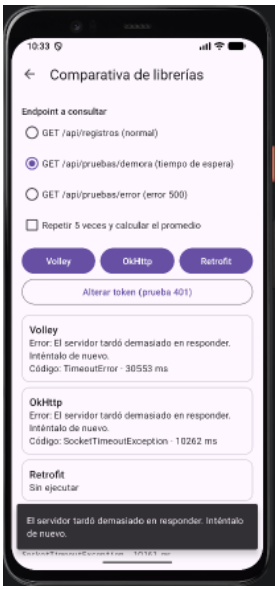


Figura 15. OkHttp: SocketTimeoutException, 10 262 ms



Figura 16. Retrofit: tiempo agotado

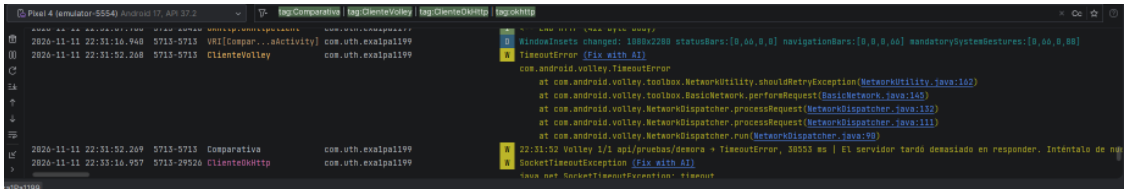


Figura 17. Logcat: TimeoutError de Volley

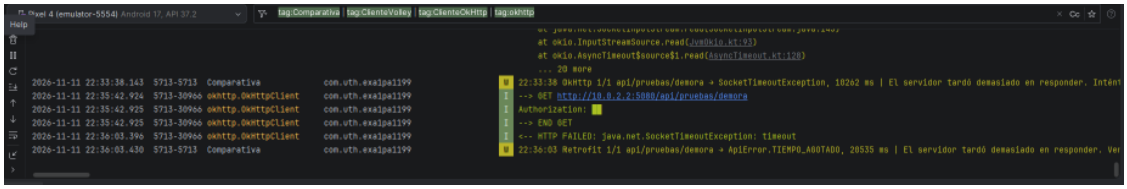


Figura 18. Logcat: OkHttp a los 10 s y Retrofit a los 20 s

10.4 Token alterado (401)

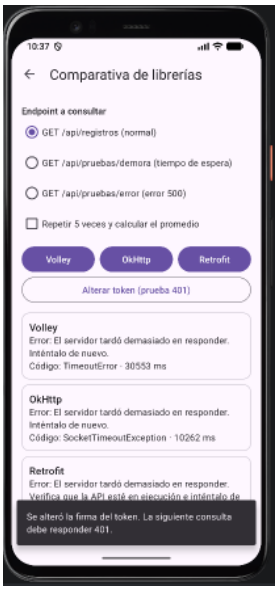


Figura 19. Firma del token alterada

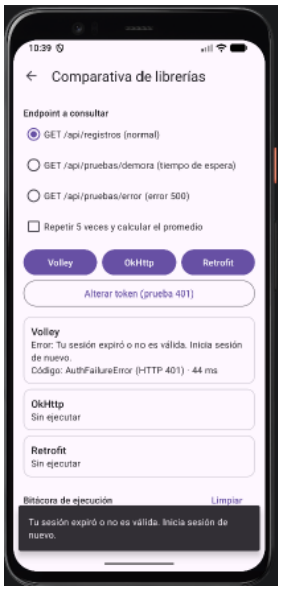


Figura 20. Volley: AuthFailureError 401

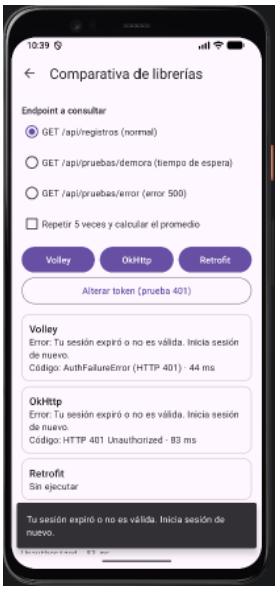


Figura 21. OkHttp: HTTP 401

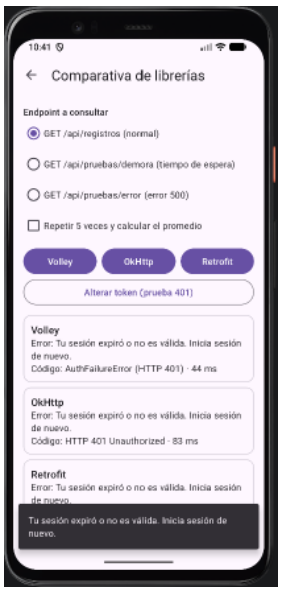


Figura 22. Retrofit: sesión no válida

10.5 Error interno del servidor (500)

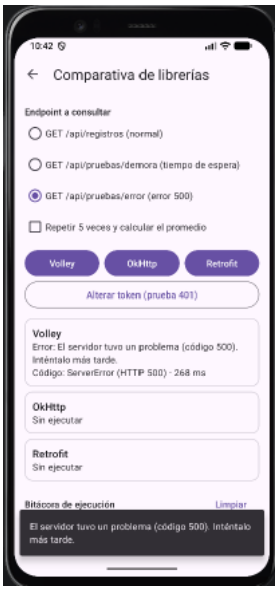


Figura 23. Volley: ServerError 500

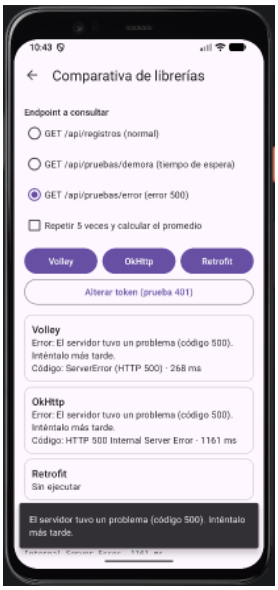


Figura 24. OkHttp: HTTP 500

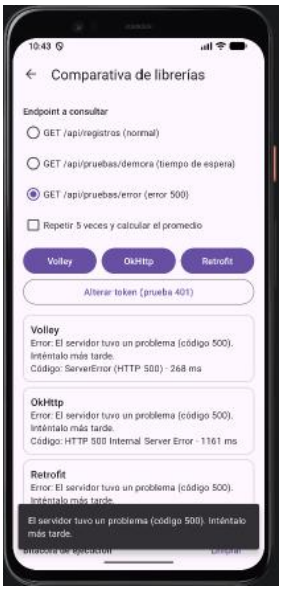


Figura 25. Retrofit: ApiError 500

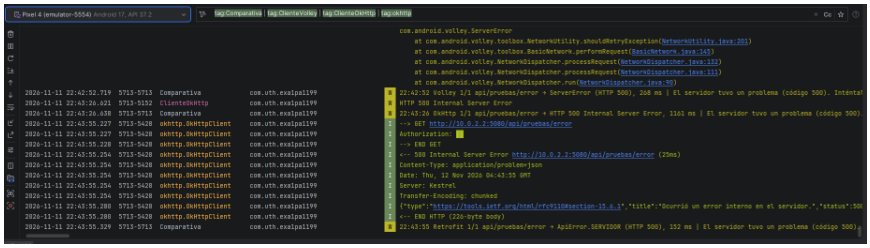


Figura 26. Logcat: las tres librerías reciben el 500 con ProblemDetails

11. CUADRO COMPARATIVO

Tabla 7. Comparación de las tres librerías

Criterio	Volley	OkHttp	Retrofit
Líneas de código de la consulta	69	65	24, más la infraestructura reutilizada de la 2.3 (interfaz, ApiClient y repositorio)
Envío de encabezados	Sobrescribir getHeaders() en cada petición	header() en cada Request.Builder	Una vez en AuthInterceptor para todas las peticiones
Conversión de JSON	Entrega JSONArray; conversión manual o con Gson	Entrega texto; conversión manual con Gson	Automática con GsonConverterFactory
Hilo de la respuesta	Hilo principal	Hilo de fondo; requiere runOnUiThread	Hilo principal

Criterio	Volley	OkHttp	Retrofit
Manejo de errores	Subclases de VolleyError; 401 y 500 llegan como error	4xx y 5xx llegan a onResponse; solo los fallos de red a onFailure	Un solo punto: ApiErrorMapper convierte todo a ApiError
Reintentos y tiempos de espera	DefaultRetryPolicy por petición; reintenta automáticamente	Tiempos en el cliente; sin reintento de la petición	Usa la configuración de OkHttp del ApiClient
Prueba y mantenimiento	Difícil de probar; URLs y encabezados repetidos	Más control, pero más código repetido	Interfaz declarativa, fácil de simular en pruebas y de mantener
Promedio de 5 ejecuciones	311.8 ms	242.4 ms	244.6 ms

12. RECOMENDACIÓN

Para el proyecto final se recomienda Retrofit. Aunque OkHttp obtuvo el menor promedio, la diferencia con Retrofit fue de solo 2 ms y Retrofit usa OkHttp internamente, por lo que conserva su rendimiento y su control de tiempos de espera.

- **Menos código y menos errores.** Los endpoints se declaran en una interfaz y la conversión de JSON es automática; agregar un endpoint nuevo requiere pocas líneas.
- **Seguridad centralizada.** El token se agrega en un solo interceptor y el 401 cierra la sesión en un solo lugar.
- **Errores uniformes.** Todas las fallas se convierten en ApiError, lo que da al usuario mensajes consistentes.
- **Hilo correcto.** La respuesta llega en el hilo principal, sin riesgo de actualizar la interfaz desde otro hilo.
- **Ya está integrado.** El proyecto usa Retrofit desde la tarea 2.3 con MVVM y repositorio.

Volley queda como opción para peticiones simples y casos donde el reintento automático sea deseable, y OkHttp directo se justifica cuando se necesita control de bajo nivel, por ejemplo descargas o WebSockets.

13. CONCLUSIONES

- Las tres librerías consumieron correctamente el endpoint protegido enviando el token JWT en el encabezado Authorization.
- La diferencia más importante para el código es el hilo de respuesta: OkHttp responde en un hilo de fondo y obliga a usar runOnUiThread, mientras Volley y Retrofit responden en el hilo principal.
- Cada librería clasifica los errores de forma distinta; Volley trata 401 y 500 como errores, OkHttp solo los fallos de red, y Retrofit con el repositorio los unifica en ApiError.
- Los tiempos de espera y reintentos deben configurarse a propósito: con un reintento, Volley tardó 30 s en informar la falla frente a 10 s de OkHttp.

- El manejo de errores permitió que la aplicación mostrara mensajes claros sin cerrarse en ninguno de los cinco escenarios.